```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

coordinates = np.array([
    [50, 50],
    [20, 60],
    [30, 80],
    [60, 90],
    [80, 70],
    [90, 40],
    [70, 20],
    [40, 30],
    [15, 25],
    [55, 55],
    [85, 85]
])

demands = np.array([0, 10, 15, 12, 8, 14, 9, 11, 7, 13, 6])

vehicle_capacity = 30
num_ants = 20
iterations = 150

alpha = 1.0
beta = 3.0
evaporation = 0.5
Q = 100

num_nodes = len(coordinates)

distance = np.zeros((num_nodes, num_nodes))

for i in range(num_nodes):
    for j in range(num_nodes):
        distance[i][j] = np.linalg.norm(
            coordinates[i] - coordinates[j]
        )

pheromone = np.ones((num_nodes, num_nodes))

heuristic = 1 / (distance + 1e-10)
np.fill_diagonal(heuristic, 0)

def route_distance(route):
    total = 0
    for i in range(len(route) - 1):
        total += distance[route[i]][route[i + 1]]
    return total

def construct_solution():
    unvisited = set(range(1, num_nodes))
    routes = []

    while unvisited:
        route = [0]
        current = 0
        load = 0

        while True:
            candidates = [
                city for city in unvisited
                if load + demands[city] <= vehicle_capacity
```

```
            ]

            if not candidates:
                break

            probabilities = []

            for city in candidates:
                value = (
                    pheromone[current][city] ** alpha
                    * heuristic[current][city] ** beta
                )
                probabilities.append(value)

            probabilities = np.array(probabilities)
            probabilities /= probabilities.sum()

            next_city = np.random.choice(candidates, p=probabilities)

            route.append(next_city)
            load += demands[next_city]
            unvisited.remove(next_city)
            current = next_city

        route.append(0)
        routes.append(route)

    return routes

def total_distance(routes):
    return sum(route_distance(route) for route in routes)

best_routes = None
best_distance = float("inf")
convergence = []

for iteration in range(iterations):
    ant_solutions = []

    for ant in range(num_ants):
        routes = construct_solution()
        cost = total_distance(routes)

        ant_solutions.append((routes, cost))

        if cost < best_distance:
            best_distance = cost
            best_routes = [r.copy() for r in routes]

    pheromone *= (1 - evaporation)

    for routes, cost in ant_solutions:
        deposit = Q / cost

        for route in routes:
            for i in range(len(route) - 1):
                a = route[i]
                b = route[i + 1]

                pheromone[a][b] += deposit
                pheromone[b][a] += deposit

    for route in best_routes:
        for i in range(len(route) - 1):
            a = route[i]
            b = route[i + 1]

            pheromone[a][b] += Q / best_distance
```

```
            pheromone[b][a] += Q / best_distance

    convergence.append(best_distance)

    if (iteration + 1) % 25 == 0:
        print(
            "Iteration:", iteration + 1,
            "| Best Distance:", round(best_distance, 2)
        )

print("\n========== OPTIMIZED CVRP RESULTS ==========")

for i, route in enumerate(best_routes):
    load = sum(demands[city] for city in route)

    print("\nVehicle", i + 1)
    print("Route:", " -> ".join(map(str, route)))
    print("Total Load:", load)
    print("Vehicle Capacity:", vehicle_capacity)
    print("Remaining Capacity:", vehicle_capacity - load)
    print("Route Distance:", round(route_distance(route), 2))

print("\n============================================")
print("Minimum Total Distance:", round(best_distance, 2))
print("Total Vehicles Used:", len(best_routes))
print("Total Customers:", num_nodes - 1)
print("============================================")

plt.figure(figsize=(10, 7))

for i, route in enumerate(best_routes):
    x = [coordinates[city][0] for city in route]
    y = [coordinates[city][1] for city in route]

    plt.plot(
        x, y,
        marker='o',
        linewidth=2,
        markersize=7,
        label=f'Vehicle {i+1}'
    )

plt.scatter(
    coordinates[0][0],
    coordinates[0][1],
    color='red',
    marker='s',
    s=200,
    label='Depot',
    zorder=5
)

for i, (x, y) in enumerate(coordinates):
    label = "Depot" if i == 0 else f"C{i}"
    plt.annotate(
        label,
        (x, y),
        xytext=(5, 7),
        textcoords='offset points'
    )

plt.title("CVRP Optimization Using Ant Colony Optimization")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.show()

plt.figure(figsize=(10, 5))

plt.plot(
    range(1, iterations + 1),
    convergence,
    linewidth=2
)

plt.title("ACO Convergence Graph")
plt.xlabel("Number of Iterations")
plt.ylabel("Best Total Distance")
plt.grid(True)
plt.tight_layout()
plt.show()

vehicle_distances = [
    route_distance(route) for route in best_routes
]

plt.figure(figsize=(9, 5))

plt.bar(
    [f'Vehicle {i+1}' for i in range(len(best_routes))],
    vehicle_distances
)

plt.title("Distance Travelled by Each Vehicle")
plt.xlabel("Vehicles")
plt.ylabel("Route Distance")
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```